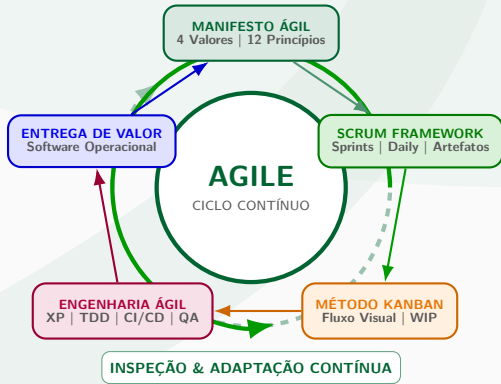


Agilidade e Processo de Software

Metodologias Ágeis, Scrum, Kanban e Práticas de Engenharia

Prof. Me. Sergio Souza Novak

Faculdade de Engenharia de Software — UniRV



1. Fundamentos & Filosofia Ágil

1 O Manifesto Ágil

- Origem e Propósito
- Valores e Princípios

2 O que é Agilidade? (Seção 3.1)

- Cultura e Colaboração
- O Kanban Ágil
- Histórias de Usuário
- Simulação Prática (Azure DevOps)
- O Papel do Scrum Master
- Paralelismo no Desenvolvimento
- Qualidade (QA) na Sprint
- Lidando com a Imprevisibilidade

2. Frameworks, Métodos & Qualidade

3 Agilidade e o Custo das Mudanças (Seção 3.2)

- Achatamento da Curva

4 O Processo Ágil

- Os 12 Princípios

5 Scrum

- Fundamentos e Artefatos
- Cerimônias Scrum (Eventos)
- Aprofundamento: Casos de Teste

6 Conceito e Filosofia Ágil

7 Equipe e o Fator Humano

- O Papel da Equipe
- O Fator Humano (Cockburn)

8 Processo Ágil e Artefatos

- Etapas e Artefatos

9 Aplicabilidade e Limitações

O Marco Histórico

Em **2001**, um grupo composto por 17 desenvolvedores de software, autores e consultores de renome [Beck et al., 2001] assinou o “*Manifesto para o Desenvolvimento Ágil de Software*”.

Propósito

Propor uma alternativa inovadora aos modelos de processo prescritivos e rígidos, sanando fraquezas da engenharia de software convencional.

- **Contexto:** Ambientes acelerados demandavam maior flexibilidade.
- **Foco:** Valorizar as pessoas, a entrega contínua de valor e a adaptação a mudanças.

Os Quatro Valores do Manifesto Ágil

O Manifesto Ágil defende quatro pilares fundamentais:



Nota: Embora haja valor nos itens à direita, valoriza-se mais os itens à esquerda.

O que é Agilidade no Contexto de Software?

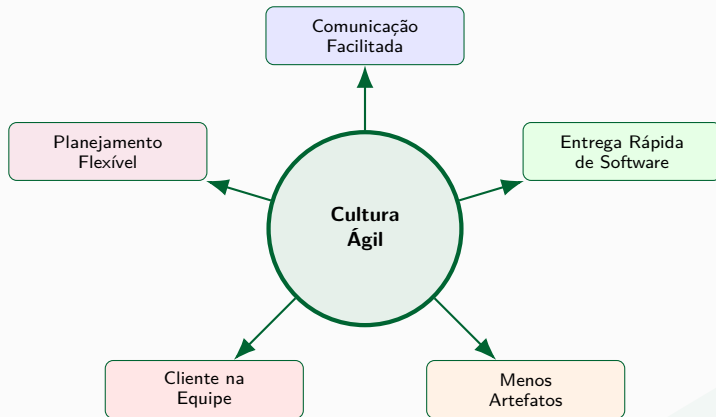
O Condutor da Agilidade (Jacobson)

Ivar Jacobson [Jac02a] argumenta que a **difusão da mudança** é o principal condutor para a agilidade.

- Engenheiros de software devem ser rápidos para assimilar rápidas mudanças no ambiente.

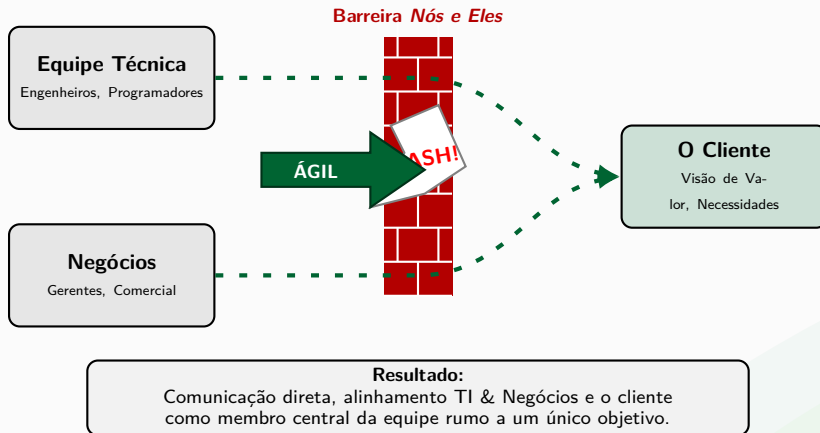
Além da Simples Resposta à Mudança

Agilidade abrange uma **filosofia e cultura** de trabalho estruturada em diversos pilares, que serão detalhados a seguir: comunicação, foco em entrega, redução de artefatos intermediários e planejamento flexível.



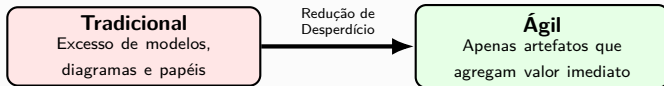
Facilitação da Comunicação e Colaboração

A agilidade **quebra barreiras** e aproxima todos os envolvidos no projeto, substituindo a atitude “*nós e eles*” pelo “*somos uma equipe*”.



Software Operacional Acima de Tudo

A ênfase primária é a **entrega rápida e contínua** de software que funciona, em vez de documentação extensa.

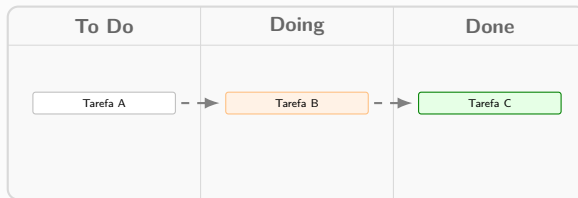


- **Exemplo:** Em vez de criar um documento de requisitos de 100 páginas, a equipe ágil mantém um conjunto priorizado de *Histórias de Usuário*.

Transparência e Fluxo: O Quadro Kanban

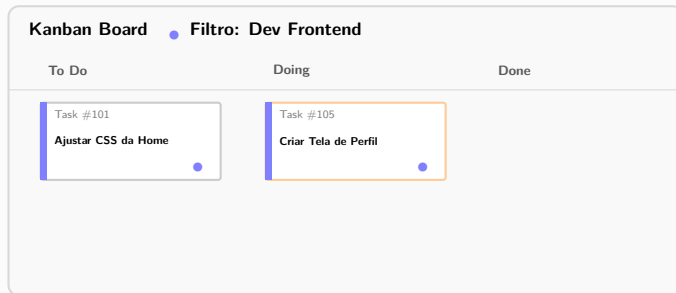
Para organizar o trabalho ágil, as equipes utilizam quadros visuais, como o **Kanban**, que mapeiam o ciclo de vida das tarefas desde a sua criação até a conclusão.

- **To Do (A Fazer):** Tarefas mapeadas para a Sprint atual, aguardando início.
- **Doing (Em Andamento):** Onde os desenvolvedores estão atuando ativamente.
- **Done (Concluído):** Tarefas finalizadas e testadas, prontas para entrega.



Quadro Kanban: Visão do Desenvolvedor Frontend

O desenvolvedor **Frontend** foca nas tarefas relacionadas à interface com o usuário, telas, estilos e consumo de APIs.



Quadro Kanban: Visão do Desenvolvedor Backend

Enquanto isso, o desenvolvedor **Backend** visualiza e atua nas tarefas de infraestrutura, banco de dados e regras de negócio.



Exemplo Prático: História de Usuário (User Story)

Em ferramentas como o **Azure DevOps**, os requisitos não são documentos gigantescos, mas sim cartões chamados de *User Stories*.

1234

Tela de Login Seguro

New ▾

Save & Close

Description

Como um usuário do sistema,
Eu quero fazer login usando autenticação em dois fatores,
Para que meus dados estejam protegidos.

Acceptance Criteria

- ☐ O sistema deve enviar SMS com código de 6 dígitos.
- ☐ O código expira em 5 minutos.
- ☐ Bloqueio após 3 tentativas falhas.

Details

Assignee

Unassigned

Story Points

5

Area

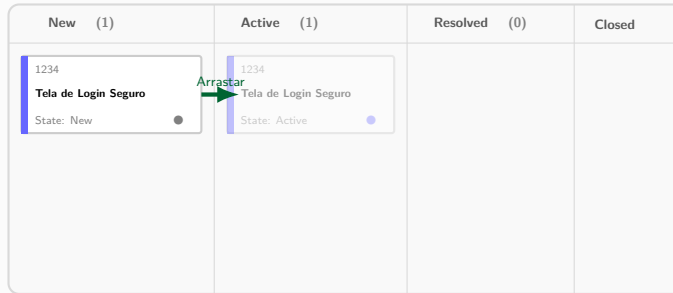
Frontend\Auth

Iteration

Sprint 14

Simulação: Kanban Board (Azure Boards)

A História de Usuário é movida no quadro Kanban à medida que a equipe trabalha, garantindo total transparência do progresso.



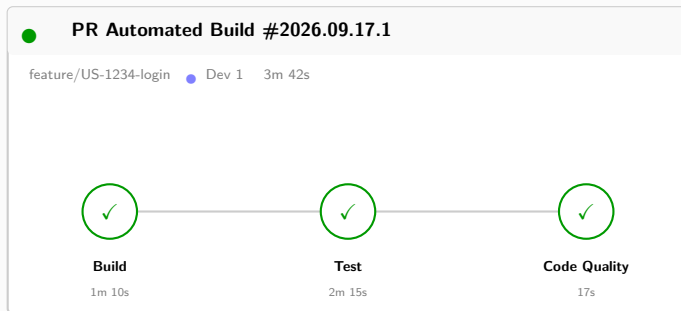
Simulação: Código e Versionamento (Azure Repos)

Ao iniciar a tarefa, o desenvolvedor cria um **Ramo (Branch)** diretamente vinculado à História de Usuário.

Commits (Branch) feature/US-1234-login ▾		
Message	Author	Commit
Added 2FA logic to auth controller Linked to: #1234 Tela de Login Seguro	● Dev 1	a1b2c3d
Implemented OTP input UI components Linked to: #1234 Tela de Login Seguro	● Dev 1	f4e5d6c
Merge pull request #54 from main	● Dev 2	7a8b9c0

Simulação: Integração Contínua (Azure Pipelines)

Quando o código está pronto, o Dev cria um **Pull Request** (PR). Isso aciona um *Pipeline* automatizado para compilar e testar a solução.



Simulação: Revisão e Fechamento (Pull Request)

Outro membro da equipe revisa o código, aprova e o *Merge* (união com o ramo principal) é realizado, concluindo a História.

PR 55: Adiciona autenticação de dois fatores (2FA)Complete

Active ● Dev 1 requested into main from feature/US-1234-login

Files
+ 124 lines - 12 lines

Linked Work Items
■ 1234 Tela de Login Seguro

Reviewers
✓ ● Dev 2 (Approved)

Checks
✓ Build succeeded
✓ Work item linked
✓ Comments resolved (3/3)

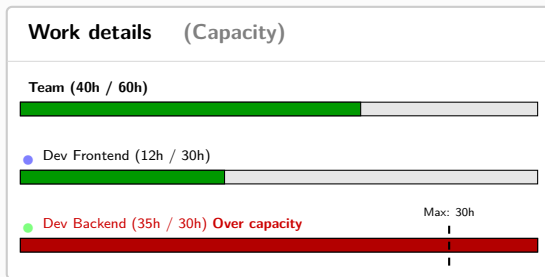
Papel do Scrum Master: Visão Geral da Sprint

O **Scrum Master** acompanha a saúde da Sprint analisando o total de Histórias de Usuário, tarefas (Tasks) e a carga de trabalho de toda a equipe.

Sprint 14		Aug 1 - Aug 15		View: Backlog ▾	
Title		State	Effort	Assigned To	
	1234 Tela de Login Seguro	Active	5		Unassigned
	1235 Criar interface (Frontend)	Active	12h		Dev Frontend
	1236 API de Autenticação (Backend)	Closed	8h		Dev Backend
	1280 Integração com Gateway de Pagto	New	8		
	1281 Consumir webhook de pagto	New	20h		Dev Backend

Papel do Scrum Master: Gestão de Capacidade

O Scrum Master nota um problema crítico no Azure DevOps: a barra de capacidade do Dev Backend está vermelha! O dev tem mais horas estimadas em tarefas do que horas disponíveis na Sprint.



Papel do Scrum Master: Protegendo o Time

Para resolver a sobrecarga, o Scrum Master e o *Product Owner* movem histórias menos prioritárias para a próxima Sprint (ou para o Backlog), protegendo o time.



A agilidade permite que diferentes especialidades atuem ao mesmo tempo. O **Dev Frontend** filtra o quadro Kanban apenas para suas tarefas.



Paralelismo: Visão do Dev Backend

No mesmo exato momento, o **Dev Backend** está focado nas suas atividades da mesma História, garantindo que o tempo da equipe seja otimizado.

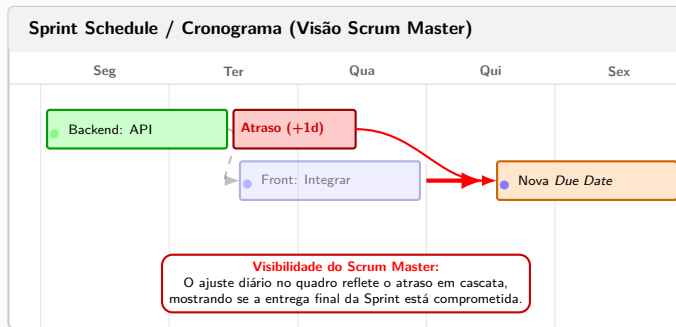
Taskboard ● **Filtered: Dev Backend**

To Do	Doing	Done
	Task 1236 API de Autenticação / JWT Remaining: 8h ●	

Criando os *endpoints* da API em paralelo com o trabalho do Frontend.

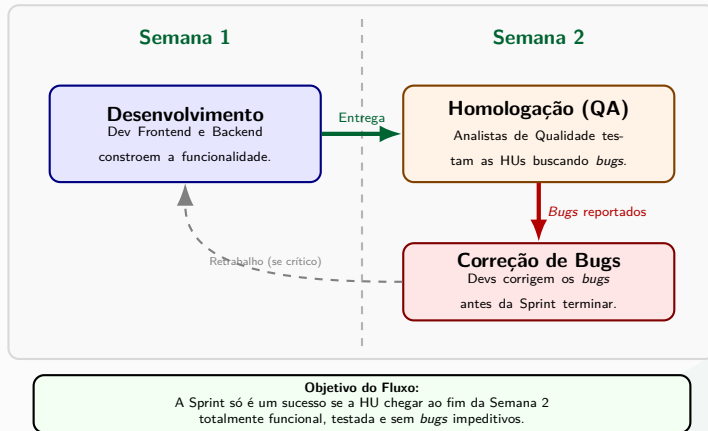
Daily Scrum: Visibilidade e Ajuste de Prazos

Na **Daily Scrum**, a equipe sincroniza o andamento. Se uma tarefa (ex: Backend) atrasar, as *due dates* (datas de entrega) das demais são ajustadas, dando visibilidade imediata ao Scrum Master.

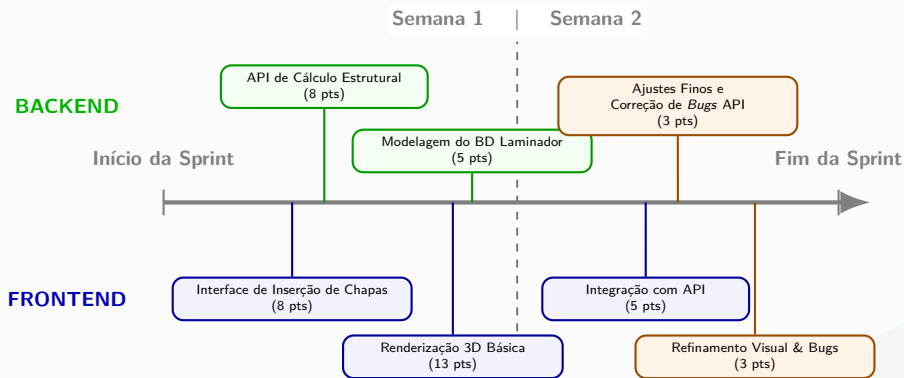


Fluxo da Sprint: Desenvolvimento e Qualidade (QA)

Em uma Sprint típica de **duas semanas**, o trabalho é cadenciado para garantir que a História de Usuário (HU) seja desenvolvida, testada e corrigida dentro do mesmo ciclo.



Exemplo Prático: Módulo de Construção de Laminador



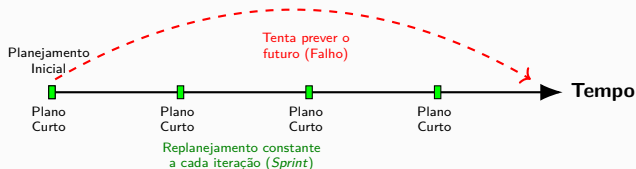
A agilidade reconhece os limites do planejamento a longo prazo.

O Plano Rígido

Assumir que os requisitos não mudarão gera projetos propensos a falhas, atrasos e alto custo de mudança quando o cliente alterar de ideia.

O Plano Flexível (Ágil)

O planejamento é contínuo e adaptativo. Aceita-se que o escopo pode e **vai** mudar ao longo do desenvolvimento.



Requisitos para Projetar um Processo Ágil

A agilidade pode ser applied a **qualquer** processo de software, desde que seja projetado para que a equipe possa:

Pilares de Projeto do Processo Ágil

- ❶ **Adaptar e racionalizar tarefas:** Ajustar o trabalho à realidade do projeto;
- ❷ **Planejamento fluido:** Compreender a natureza dinâmica da metodologia;
- ❸ **Artefatos enxutos:** Eliminar tudo, exceto os artefatos essenciais;
- ❹ **Entrega incremental:** Entregar software operacional ao cliente o mais rápido possível.

Alerta: Agilidade não é Licença para Gambiarra

Não cometa o erro de supor que a agilidade dá licença para abreviar soluções!

Processo é um requisito, e a disciplina é essencial.

O Senso Comum na Engenharia de Software

Décadas de experiência demonstram que os **custos de mudanças aumentam de forma não linear** (exponencial) conforme o projeto avança.

Mudança no Início (Requisitos)

- **Ações:** Editar especificação, alterar detalhamento de uso ou ampliar lista de funções.
- **Custo:** Mínimo.
- **Impacto:** Praticamente nulo no cronograma final.

Mudança Tardia (Validação)

- **Ações:** Alterar arquitetura, criar 3 novos componentes, modificar 5 existentes, novos testes.
- **Custo:** Cresce rapidamente.
- **Impacto:** Esforço massivo para evitar efeitos colaterais.

Exemplo Prático: Impacto de uma Mudança Tardia

Se uma grande mudança funcional for solicitada durante os **testes de validação**:

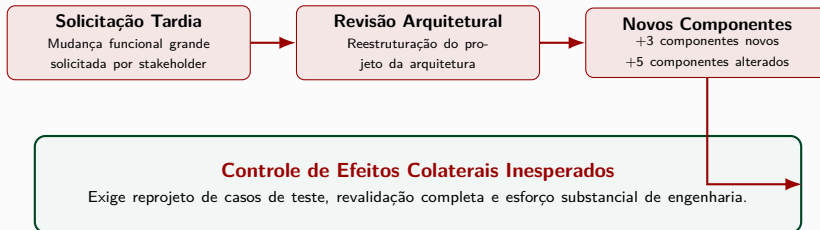
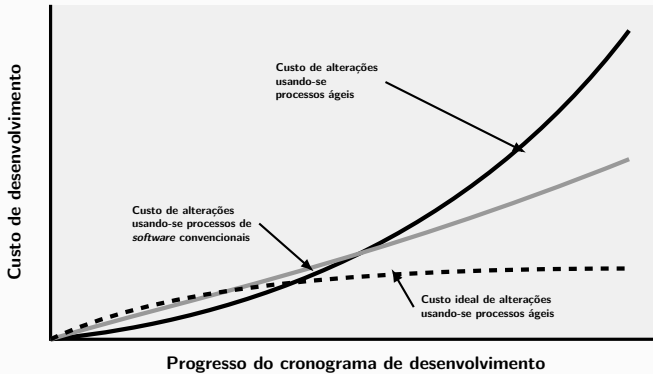


Figura 3.1 – Curva do Custo das Mudanças



Achatamento da Curva de Mudança

O Argumento Ágil

Um processo ágil bem elaborado **“achata” o custo da curva de mudança**, permitindo assimilar alterações tardias sem impacto significativo.

Como isso é alcançado?

- **Entregas Incrementais:** Reduzem o escopo da mudança.
- **Práticas Ágeis Associadas:** Testes contínuos de unidades e programação em pares.

Debate em Curso

Há evidências de redução significativa nos custos de alterações, mas o nível exato em que a curva se torna plana ainda é debatido.

Um processo ágil lida com a imprevisibilidade lidando com três preceitos fundamentais [Fow02]:

- ❶ **Dificuldade de Previsão:** É difícil prever quais requisitos persistirão e como as prioridades do cliente sofrerão alterações.
- ❷ **Projeto e Construção Intercalados:** Modelos de projeto precisam ser testados durante a construção. É difícil saber quanto de projeto é necessário antes da implementação.
- ❸ **Planejamento Limitado:** Análise, projeto, construção e testes não são tão previsíveis do ponto de vista de planejamento.

Adaptabilidade do Processo

Para administrar a imprevisibilidade, o processo deve ser **adaptável** para alterar rapidamente o projeto e as condições técnicas.

Adaptação Incremental e Feedback

- A equipe precisa do **feedback do cliente** para fazer adaptações apropriadas.
- O **protótipo operacional** (ou incremento executável) é o catalisador eficaz para esse feedback.
- Entregas curtas capacitam o cliente a avaliar o incremento e influenciar o processo regularmente.

A satisfação do cliente é alcançada com a **oferta de valor e entregas frequentes**.

- **Equipe e Ambiente:** Indivíduos motivados, comunicação presencial e auto-organização.
- **Foco Técnico:** Excelência técnica e bom projeto.
- **Simplicidade:** A arte de maximizar o volume de trabalho *não realizado*.
- **Software Operacional:** O objetivo principal, desenvolvido em um ritmo sustentável a longo prazo.
- **Introspecção e Melhoria:** A equipe busca continuamente melhorar sua forma de trabalho.

O debate metodológico entre agilistas e tradicionais não deve virar uma "guerra santa".

Visão Ágil Extremista

Os metodologistas tradicionais focam em documentação excessiva em vez de sistemas úteis e operacionais.

Visão Tradicional Extremista

Os ágeis são "hackers" que terão surpresas ao lidar com softwares de porte empresarial.

O Caminho do Meio

Software funcional é importante, mas também precisa de confiabilidade, usabilidade e facilidade de manutenção. Deve-se **mesclar o melhor das duas escolas**.

O Método Scrum

O que é?

Scrum (nome derivado do *rugby*) é um método ágil concebido por Jeff Sutherland (início dos anos 1990) e expandido por Schwaber e Beedle.

Princípios e Atividades

Coerente com o manifesto ágil, o Scrum incorpora atividades de **requisitos, análise, projeto, evolução e entrega**.

Sprints

O trabalho ocorre em janelas de tempo limitadas (*time-boxes*) chamadas **sprints**. O número de sprints varia conforme o projeto, e o processo é adaptado pela equipe em tempo real.

A Equipe Scrum

É interdisciplinar, auto-organizada e pequena (3 a 6 desenvolvedores), composta por:

- **Product Owner:** Cuida dos interesses do cliente/negócio.
- **Scrum Master:** Facilitador que remove obstáculos.
- **Equipe de Desenvolvimento.**

O Backlog do Produto

Uma **lista priorizada** de requisitos que agregam valor ao negócio. É dinâmico e ordenado pelo *Product Owner*.

O Backlog do Sprint

Subconjunto do backlog do produto selecionado para ser completado no *sprint* atual (geralmente 2 a 4 semanas). Não se adiciona novas características durante o sprint.

O Incremento de Código

União de itens completados nos sprints anteriores com os itens do sprint atual. A equipe decide como completá-lo e quando está pronto.

O Papel do Scrum Master

Comanda a reunião diária de 15 minutos, remove obstáculos, ajuda a equipe e o *product owner* a gerenciar o backlog.

Reunião de Planejamento do Sprint

Preparação e Priorização

A equipe e o *product owner* desenvolvem e ordenam os itens do backlog pela importância para o negócio e pela complexidade técnica.

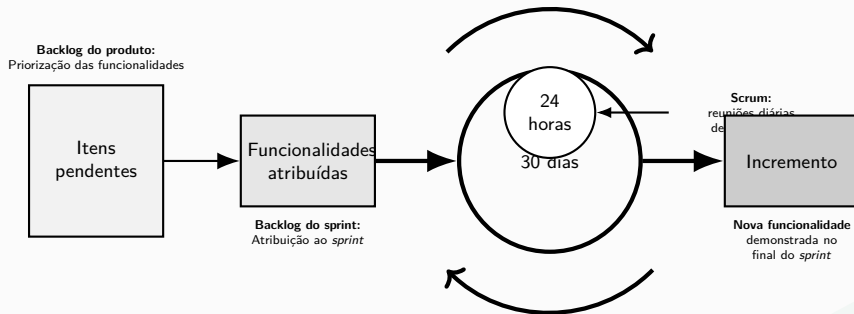
Definindo a Meta do Sprint

Antes do sprint, o *product owner* enuncia a **meta de desenvolvimento**. O Scrum Master e a equipe selecionam os itens para o *backlog do sprint*.

Comprometimento da Equipe

A equipe determina o que pode entregar no incremento dentro da janela de tempo e define os papéis necessários para a execução do trabalho.

Figura 3.2 – Fluxo do Processo Scrum



O que é?

Evento diário de **15 minutos** para sincronizar atividades e planejar as próximas 24 horas. Participam obrigatoriamente o *Scrum Master* e a equipe de desenvolvimento.

Microsoft Teams | Canal: Daily Scrum

SM

Scrum Master 09:00 AM

Bom dia, time! Hora da nossa Daily. Por favor, respondam às **3 Perguntas-Chave** para nosso alinhamento diário.

DF

Dev Frontend 09:05 AM

1. O que você realizou desde a última reunião?
- Finalizei a interface da tela de Login e enviei o PR.
2. Quais obstáculos está encontrando?
- Sem impedimentos técnicos no momento.
3. O que planeja realizar até a próxima reunião?
- Iniciar a listagem de usuários e consumir a API do Backend.

Objetivos

Revelar problemas cedo. O Scrum Master deve focar em **eliminar os obstáculos** antes do próximo encontro. Não é uma reunião para solucionar problemas (isso é feito à parte). Promove a "socialização do conhecimento".

Reunião de Revisão do Sprint

Quando e Quem?

Ocorre ao final do sprint (ex: 4 horas de reunião para sprint de 4 semanas). Participam: Scrum Master, Equipe de Desenvolvimento, Product Owner e envolvidos.

A Demonstração (Demo)

A atividade principal é uma **demonstração do incremento de software** completado (funções entregues na janela de tempo, não necessariamente tudo o que foi planejado).

Aceitação e Feedback

O *Product Owner* pode ou não aceitar o incremento. Feedbacks são recolhidos para a próxima rodada de planejamento, onde novas características podem ser adicionadas ou eliminadas do backlog.

Retrospectiva do Sprint

Objetivo e Duração

Realizada após a Revisão e antes do próximo Planejamento (ex: 3 horas para sprint de 4 semanas).

Debate da Equipe

O Scrum Master encoraja a equipe a debater e responder:

- O que deu certo no sprint?
- O que poderia melhorar?
- Com o que a equipe se compromete a melhorar no próximo sprint?

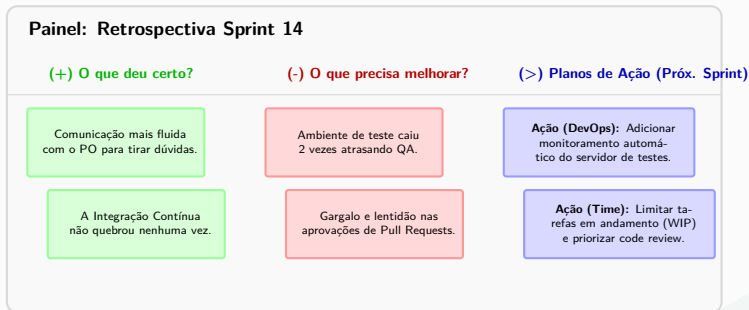
Melhoria Contínua

A equipe busca melhorar suas práticas, adaptar a definição de "pronto" e se preparar efetivamente para o próximo incremento.



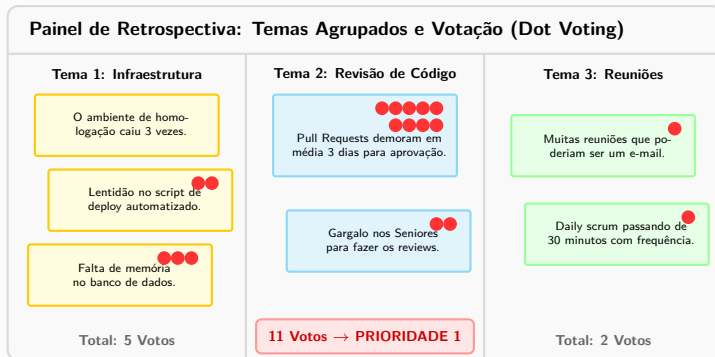
Exemplo Prático: Pauta de Retrospectiva

Uma ferramenta visual colaborativa (como *Miro* ou *Trello*) é frequentemente usada para estruturar o debate e agrupar os *post-its* virtuais da equipe.



Retrospectiva em Equipes Grandes: Agrupamento e Votação

Em equipes maiores, dezenas de tópicos são levantados simultaneamente. Para focar no que é mais crítico, a equipe **agrupa os post-its por temas similares** e utiliza **Dot Voting** (votação por pontos) para priorizar a discussão e definir planos de ação.



O que é um Caso de Teste?

É um conjunto de **condições**, **passos** e **variáveis** sob as quais um Analista de Qualidade (QA) ou testador determinará se um requisito de software (como uma História de Usuário) foi implementado corretamente e funciona conforme esperado.

- **No Azure DevOps:** São representados como *Work Items* específicos (geralmente organizados dentro de um "Test Plan").
- **Composição Básica:** Possuem *Pré-condições*, *Ações (Passos a executar)*, e o *Resultado Esperado* para cada passo.
- **Rastreabilidade:** Ficam vinculados diretamente à História de Usuário que estão validando. Se a User Story muda, o Caso de Teste deve ser revisado e atualizado.

A execução de um Caso de Teste no ciclo ágil garante que novas funcionalidades entreguem o valor esperado e não quebrem o restante do sistema.

- **Manual vs Automatizado:** Podem ser executados manualmente por um humano (clikando e validando visualmente) ou via scripts automatizados (ex: Cypress, Selenium, JUnit).
- **Passo a Passo (Test Runner):** Em ferramentas como Azure DevOps, o QA usa um painel chamado *Test Runner* que exibe o passo a passo na tela. Para cada passo, ele marca como **Passou (Passed)** ou **Falhou (Failed)**.
- **Evidências e Bugs:** Se um passo falha, o testador anexa evidências (prints de tela, logs de rede) e abre um *Bug* diretamente vinculado àquele passo que falhou, bloqueando a entrega final da User Story até a correção.

Criação pelo Analista de Qualidade (QA)

Durante o planejamento ou no decorrer da Sprint, o Analista de Qualidade cria os cenários positivos e negativos para cobrir todas as regras de negócio da História.

Azure DevOps | **Test Plans** > **Test Case 305**

Validar Login com credenciais incorretas ☐ Result

State: Design Assigned To: Analista QA Area: Autenticação

Step	Action	Expected Result	Pass?
1	Acessar a tela de login (/login).	Campos de usuário e senha visíveis. ☐	
2	Inserir email "teste@errado" e senha "123".	Botão 'Entrar' habilitado. ☐	
3	Clicar no botão 'Entrar'.	Exibir alerta vermelho: "Credenciais inválidas". ☐	

Casos focados em garantir que a **Interface de Usuário (UI)** reaja como esperado (responsividade, estados de botões, validação visual e usabilidade).

Azure DevOps | Test Plans > Test Case 308

Comportamento responsivo do menu lateral (Frontend) ☐ Result

Assigned To: Dev Frontend Linked Work Item: User Story 105

Step	Action (UI)	Expected Result	Pass?
1	Abrir aplicação no navegador Chrome desktop.	Menu lateral expandido ocupando 250px.	
2	Redimensionar janela para largura de 320px.	Menu lateral retrai e vira ícone hambuguer.	
3	Clicar no ícone hamburguer.	Menu lateral sobrepõe a tela (overlay).	

Casos focados em garantir que as **Regras de Negócio e APIs** (Endpoints, payloads, validações de segurança e persistência no banco de dados) funcionem corretamente.

Azure DevOps | Test Plans > Test Case 310

Validação de Payload no Endpoint POST /users (Backend) ☐ Result

Assigned To: Dev Backend Linked Work Item: User Story 106

Step	Action (API / Postman)	Expected Result	Pass?
1	Enviar POST /users com payload sem campo 'email'.	Retornar HTTP 400 (Bad Request).	☐
2	Validar o body da resposta de erro JSON.	Body deve conter: "Email is required".	☐
3	Verificar integridade do banco de dados.	Nenhum usuário foi criado na tabela 'users'.	☐

O que é a Engenharia de Software Ágil?

A Engenharia de Software Ágil combina uma **filosofia** com um conjunto de **princípios de desenvolvimento**.

A Filosofia Ágil

- Satisfação do cliente por meio da **entrega incremental antecipada**;
- Equipes de projeto **pequenas e altamente motivadas**;
- Utilização de **métodos informais** e artefatos mínimos;
- Acima de tudo: **simplicidade** no desenvolvimento geral.

Princípios de Desenvolvimento

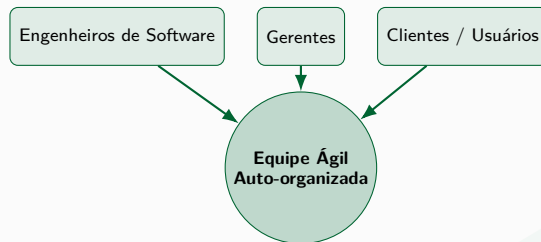
- Priorizam a **entrega** mais do que a análise e o projeto (embora essas atividades não sejam desencorajadas).

Quem Realiza? A Equipe Ágil

Engenheiros de software, gerentes, clientes e usuários trabalham conjuntamente em uma **equipe ágil**.

Características Fundamentais

- **Auto-organização:** A equipe controla seu próprio destino e toma suas decisões.
- **Comunicação Acelerada:** Elimina barreiras e aproxima todos os participantes.
- **Colaboração Contínua:** Todos os envolvidos atuam integrados a serviço do produto.



O Fator Humano nos Processos (Cockburn)

Alistair Cockburn [Cockburn, 2002] destaca uma falha essencial dos modelos prescritivos tradicionais:

Desenvolvedores Não são Robôs

Engenheiros de software apresentam grande variação individual em:

- Estilos de trabalho e níveis de habilidade;
- Criatividade, organização, consistência e espontaneidade;
- Habilidade de comunicação escrita.

Requisito para os Modelos de Processo

Para funcionarem na prática, os processos devem:

- 1 Fornecer um mecanismo realista que estimule a **disciplina** necessária; OU
- 2 Apresentar **tolerância** com as pessoas que realizam o trabalho.

Etapas do Processo: "Engenharia de Software Light"

O desenvolvimento ágil pode ser denominado como uma “**engenharia de software light**”.

- As atividades metodológicas básicas **permanecem**:



Transformação Prática

Essas fases se transformam em um **conjunto de tarefas mínimas** que impulsiona a equipe para o desenvolvimento e para a entrega contínua.

Artefatos e Garantia de Qualidade

O Artefato Principal

O artefato mais importante é um **“incremento de software” operacional**, entregue ao cliente na data combinada.

Documentos Mais Importantes

- Histórias de Usuário (*User Stories*);
- Casos de Teste Relacionados.

Garantia de Qualidade

Se a equipe concorda que o processo funciona e produz **incrementos passíveis de entrega** que **satisfaçam o cliente**, o trabalho foi realizado corretamente.

Aplicabilidade e Limitações dos Métodos Ágeis

Objetivo dos Métodos Ágeis

Sanar fraquezas, supostas e reais, da engenharia de software convencional, entregando sistemas corretos rapidamente.

Limitações Importantes

O desenvolvimento ágil oferece grandes benefícios, porém:

- **Não é indicado para todos** os projetos, produtos, pessoas e situações.

Visão Prática

Não é a antítese da prática confiável — pode ser aplicado como uma **filosofia geral** para todos os trabalhos de software.

Quando NÃO utilizar Métodos Ágeis?

Critérios de Inviabilidade

O desenvolvimento ágil pode não ser a melhor abordagem quando:

- **Requisitos Estritamente Fixos:** O escopo, o orçamento e os prazos são inflexíveis (ex: determinados contratos governamentais).
- **Sistemas Críticos:** Projetos com alto risco de vida ou regulação severa (ex: softwares aeroespaciais, equipamentos médicos), que exigem documentação rigorosa e design prévio detalhado (BDUF - *Big Design Up Front*).
- **Cliente Indisponível:** O cliente ou representante (Product Owner) não tem disponibilidade para colaboração frequente e feedback contínuo.
- **Equipes Muito Grandes ou Terceirizadas:** A comunicação orgânica (face a face) falha e a coordenação requer processos estruturados mais rígidos.
- **Cultura Organizacional Rígida:** A empresa resiste à autonomia das equipes, prefere microgerenciamento e tem baixa tolerância a mudanças de escopo.